

```
import java.util.*;

public class Customer
{
    private String customerID;
    public String name;
    private String pan;
    private Long aadhar;
    public String address;
    public int dob;
    public Long phoneNo;

    // Each customer can have multiple accounts
    ArrayList<Account> accounts = new ArrayList<>();

    // Constructor
    public Customer(String customerID, String name, String pan, Long aadhar,
int dob, Long phoneNo)
    {
        this.customerID = customerID;
        this.name = name;
        this.pan = pan;
        this.aadhar = aadhar;
        this.dob = dob;
        this.phoneNo = phoneNo;
    }

    public String getCustomerID()
    {
        return customerID;
    }
    public String getPAN()
    {
        return pan;
    }
    public Long getAadhar()
    {
        return aadhar;
    }

    public void setCustomerID(String customerID)
    {
        this.customerID = customerID;
    }
    public void setPAN(String pan)
    {
        this.pan = pan;
    }
}
```

```

    public void setAadhar(Long aadhar)
    {
        this.aadhar = aadhar;
    }

    // add account to Customer
    public void addAccount(Account acc)
    {
        accounts.add(acc);
    }

    public void checkBalance()
    {
        for(Account acc : accounts)
            acc.checkBalance();
    }

    public void updateDetails(String address, Long phoneNo)
    {
        this.address = address;
        this.phoneNo = phoneNo;
    }

    public void viewTxnHistory()
    {
        System.out.println("Transaction history feature not implemented");
    }

    public void checkCreditScore()
    {
        System.out.println("Credit Score: 750 (sample)");
    }
}

public class Account
{
    private Long accountNumber;
    public Customer customer;
    public double balance;
    public String createdOn;
    public String accountType; // individual/joint
    public String[] creditCards;

    public Account(Long accountNumber, double balance, String createdOn,
String accountType)
    {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
}

```

```
        this.createdOn = createdOn;
        this.accountType = accountType;
    }

    public Long getAccountNumber()
    {
        return accountNumber;
    }
    public void setAccountNumber(Long accountNumber)
    {
        this.accountNumber = accountNumber;
    }

    public void checkBalance()
    {
        System.out.println("Account Number:" + accountNumber + "\nBalance:" +
balance);
    }

    public void credit(double amount)
    {
        balance += amount;
        System.out.println("Amount Credited:" + amount);
    }

    public void debit(double amount)
    {
        if(balance >= amount)
        {
            balance -= amount;
            System.out.println("Amount debited:" + amount);
        }
        else
            System.out.println("Insufficient balance");
    }

    public void transfer(Account acc, double amount)
    {
        if(balance >= amount)
        {
            this.debit(amount);
            acc.credit(amount);
        }
        else
            System.out.println("Transfer failed due to insufficient balance");
    }

    public void closeAccount()
```

```

    {
        System.out.println("Account" + accountNumber + "closed");
    }

    public void createFD()
    {
        System.out.println("Fixed Deposit created");
    }
}

public class SavingsAccount extends Account
{
    public double interestRate;
    public double minBalance;

    public SavingsAccount(Long accountNumber, double balance, String
createdOn, String accountType,
                        double interestRate, double minBalance)
    {
        super(accountNumber, balance, createdOn, accountType);
        this.interestRate = interestRate;
        this.minBalance = minBalance;
    }

    // override debit
    @Override
    public void debit(double amount)
    {
        if((balance - amount) >= minBalance)
        {
            balance -= amount;
            System.out.println("Debited from Savings:" + amount);
        }
        else
            System.out.println("Minimum balance violation!");
    }
}

public class LoanAccount extends Account
{
    public double principalAmt;
    public int tenure; // months
    public double interestRate;
    public String loanType;
    public double totalOutstanding;

    public LoanAccount(Long accountNumber, double principalAmt, int tenure,
double interestRate,

```

```

        String loanType)
    {
        super(accountNumber, 0, "Today", "Loan");
        this.principalAmt = principalAmt;
        this.tenure = tenure;
        this.interestRate = interestRate;
        this.loanType = loanType;

        calculateTotalOutstanding();
    }

    public double calculateEMI()
    {
        double r = interestRate / 12 / 100;
        double emi = (principalAmt * r * Math.pow(1 + r, tenure)) /
(Math.pow(1 + r, tenure) - 1);
        return emi;
    }

    public void calculateTotalOutstanding()
    {
        totalOutstanding = calculateEMI() * tenure;
    }

    public void payEMI()
    {
        double emi = calculateEMI();
        totalOutstanding -= emi;
        System.out.println("EMI paid:" + emi);
    }

    public void payTotalOutstanding()
    {
        totalOutstanding = 0;
        System.out.println("Loan cleared");
    }
}

import java.util.*;

public class MainForBank
{
    public static void main(String args[])
    {
        ArrayList<Customer> customers = new ArrayList<>();

        // create customer

```

```

        Customer c1 = new Customer("C101", "Spandana", "PAN123", 147258369L,
2003, 987654321L);

        // create accounts
        SavingsAccount sa = new SavingsAccount(1001L, 5000, "2026-03-23",
"Individual", 4.5, 1000);
        LoanAccount la = new LoanAccount(2001L, 2200000, 48, 11.5, "Education
Loan");

        // link Account to Customer
        sa.customer = c1;
        la.customer = c1;

        // add accounts to Customer
        c1.addAccount(sa);
        c1.addAccount(la);

        // add customer to list
        customers.add(c1);

        // perform operations
        System.out.println("\n--- SAVINGS ACCOUNT OPERATIONS ---");
        sa.credit(2000);
        sa.debit(1500);
        sa.checkBalance();

        System.out.println("\n--- LOAN ACCOUNT OPERATIONS ---");
        System.out.println("EMI: " + la.calculateEMI());
        la.payEMI();
        System.out.println("Outstanding: " + la.totalOutstanding);

        // display consolidated info
        System.out.println("\n--- CUSTOMER DETAILS ---");
        for(Customer c : customers)
        {
            System.out.println("Customer ID: " + c.getCustomerID());
            System.out.println("Name: " + c.name);
            System.out.println("PAN: " + c.getPAN());
            System.out.println("Aadhar: " + c.getAadhar());

            System.out.println("\nAccounts:");
            c.checkBalance(); // shows all account balances
        }
    }
}

```

```
PS C:\Users\hp\Downloads\Programming in Java> cd "c:\Users\hp\Downloads\Programming in Java\" ; if ($?) { javac MainForBank.java } ; if ($?) { java MainForBank }
```

```
--- SAVINGS ACCOUNT OPERATIONS ---
```

```
Amount Credited:2000.0
```

```
Debited from Savings:1500.0
```

```
Account Number:1001
```

```
Balance:5500.0
```

```
--- LOAN ACCOUNT OPERATIONS ---
```

```
EMI: 57395.81959288652
```

```
EMI paid:57395.81959288652
```

```
Outstanding: 2697603.520865666
```

```
--- CUSTOMER DETAILS ---
```

```
Customer ID: C101
```

```
Name: Spandana
```

```
PAN: PAN123
```

```
Aadhar: 147258369
```

```
Accounts:
```

```
Account Number:1001
```

```
Balance:5500.0
```

```
Account Number:2001
```

```
Balance:0.0
```

```
PS C:\Users\hp\Downloads\Programming in Java> 
```